You can also pass raw SQL to the `lock` method for allowing different types of locks. For example, MySQL has an expression called `LOCK IN SHARE MODE` where you can lock a record but still allow other queries to read it. To specify this expression just pass it in as the lock option:

```
Item.transaction do
  i = Item.lock("LOCK IN SHARE MODE").find(1)
  i.increment!(:views)
end
```

If you already have an instance of your model, you can start a transaction and acquire the lock in one go using the following code:

```
item = Item.first
item.with_lock do
  # This block is called within a transaction,
  # item is already locked.
  item.increment!(:views)
end
```

12 Joining Tables

Active Record provides a finder method called `joins` for specifying `JOIN` clauses on the resulting SQL. There are multiple ways to use the `joins` method.

12.1 Using a String SQL Fragment

You can just supply the raw SQL specifying the `JOIN` clause to `joins`:

```
Client.joins('LEFT OUTER JOIN addresses ON addresses.client_id =
clients.id')
```

This will result in the following SQL:

```
SELECT clients.* FROM clients LEFT OUTER JOIN addresses ON
addresses.client_id = clients.id
```

12.2 Using Array/Hash of Named Associations

This method only works with `INNER JOIN`.

Active Record lets you use the names of the associations defined on the model as a shortcut for specifying `JOIN` clauses for those associations when using the `joins` method.

For example, consider the following `Category`, `Article`, `Comment`, `Guest` and `Tag` models:

```
class Category < ActiveRecord::Base
  has_many :articles
end

class Article < ActiveRecord::Base
  belongs_to :category
  has_many :comments
  has_many :tags
end

class Comment < ActiveRecord::Base
  belongs_to :article
  has_one :guest
end

class Guest < ActiveRecord::Base
  belongs_to :comment
end
```